



File Compressor Service System Design & Architecture

A production-minded image compression service in Node.js + TypeScript, designed to scale — with PDF compression as a second stage.

Engineering design document

Prepared as an interview-ready portfolio project

Stack: Node.js · TypeScript · sharp · BullMQ · Redis · PostgreSQL · Object Storage

Contents

1. Project Overview & Goals
2. The Core Flow — What Actually Happens
3. Decision Record — Every Choice, Justified
4. Architecture at a Glance
5. Component Breakdown
6. The Data Layer — Redis & PostgreSQL
7. Preserving Image Quality
8. Handling Scale — The 10,000-Users Question
9. Security, Validation & Abuse Prevention
10. Failure Handling & Reliability
11. Staged Build Plan
12. Free Tooling & Hosting
13. Extension — Adding PDF Compression
14. Interview Talking Points

1. Project Overview & Goals

This project is a web-based file compression service. A user uploads an image through the browser, the system compresses it while preserving good visual quality, and the user receives a download link to the smaller file. The image compressor is built first and completely; PDF compression is then added through the same pipeline as a second file type.

The design goal is not merely "make a tool that works." It is to build something that demonstrates production engineering judgment: separating concerns cleanly, reasoning about bottlenecks, handling failure, and being able to defend every technology choice. The whole system runs on free tooling and free hosting tiers.

Design principles

- **Keep the API server responsive.** The web server coordinates work; it never gets blocked doing heavy CPU compression itself.
- **Design the seams first.** Compression logic is isolated from the request handler from day one, so moving to a background-worker model is a swap, not a rewrite.
- **Ephemeral by default.** No user accounts, no stored history. Files and job state auto-expire, which is simpler to build and a genuine privacy feature.
- **Everything must be defensible.** Every choice — database, queue, format — should have a clear "why," because interviewers probe reasoning, not buzzwords.

What we deliberately leave out

No user accounts or authentication, no permanent storage of user images, and no attempt to build a distributed cluster. These are conscious scoping decisions, each with a rationale we can explain. A deployed, working, well-reasoned simple system beats a half-finished complex one.

2. The Core Flow — What Actually Happens

At its simplest, the entire service is four conceptual steps. Understanding this plain version first is essential — every advanced piece added later exists only to solve a specific problem this simple version runs into.

The simple version

- 1 The user selects an image in the browser and uploads it.
- 2 The image reaches the server.
- 3 The server runs it through **sharp** (an open-source image library), which shrinks the file.
- 4 The server returns a download link to the compressed image, which the frontend displays.

That is a complete, working compressor. The problem it runs into is in step 3: compression uses CPU time and **blocks the server**. If ten users upload at once, they queue up behind each other because the single server is busy grinding one image at a time. Under real load it falls over.

The receptionist analogy

Instead of the server doing compression itself, it acts like a **receptionist**. When an upload arrives, it writes a ticket ("compress this image"), drops it in a **basket** (the queue), and immediately hands the user a ticket number. It is now free to greet the next person. In the back room, **workers** (separate processes) grab tickets from the basket, do the actual compression, and set the result aside. When the user asks "is my ticket done?", the server checks and returns the download link once it is ready.

The production version

- 1 The user uploads an image. The API server accepts it and stores the raw file in **object storage**.
- 2 The API server creates a **job** with a unique, unguessable ID, records it, and drops the job onto a **queue**. It immediately returns the job ID to the browser — no waiting.
- 3 A **worker** process pulls the job from the queue, fetches the raw image, compresses it with sharp, and writes the compressed result back to storage.
- 4 The worker updates the job's **status** to "done" and records the download link.
- 5 The browser **polls** a status endpoint with its job ID. Once the status is "done," it receives the download link and displays the result.
- 6 After a short time (e.g. one hour), the files and job state **auto-expire** and are deleted.

3. Decision Record — Every Choice, Justified

This is the section interviewers care about most. Each major decision is recorded with what was chosen, the alternatives weighed, the reasoning, the trade-off consciously accepted, and the cost. The recurring theme: this is a free, portfolio-scale project, so choices favor defensibility and learning value over raw scale — and every trade-off is deliberate, not accidental.

DR-01 Asynchronous (queue + workers) over synchronous processing

Decision	Accept the upload, return a job ID immediately, and compress in background workers.
Alternatives considered	Synchronous processing — compress inline and return the result in the same request.
Why	Compression is CPU-bound; doing it inline blocks the server and collapses under concurrent load. Async keeps the API responsive, absorbs traffic spikes, and is the architecture that demonstrates production judgment.
Trade-off accepted	More moving parts (a queue, workers, a polling flow) and more complexity to build and reason about.
Cost	Free — Redis and worker processes cost nothing on free tiers. The real cost is engineering time and conceptual overhead.

DR-02 Build the simple version first, then re-architect

Decision	Ship a synchronous compressor first, feel it block, then move to the async design.
Alternatives considered	Jump straight to the full async architecture from day one.
Why	Building simple first means you experience the blocking bottleneck yourself, so the queue/worker design lands as a solution to a real, observed problem — the single most convincing interview story.
Trade-off accepted	Some rework: the request path changes when compression moves to a worker.
Cost	Free, and minimal rework <i>if</i> the compression logic is isolated in its own function from the start — the reason the "design the seams first" rule exists.

DR-03 Node.js + TypeScript for the API layer

Decision	Node with TypeScript for the request-handling / coordination layer.
Alternatives considered	Go, Python, or Java for the API server.
Why	The API layer's work is I/O-bound (issuing job IDs, small reads/writes), which is exactly Node's strength. TypeScript adds type safety. It is also the stack the builder is targeting.
Trade-off accepted	Node is single-threaded, so CPU-bound compression must be kept off the request thread — a constraint that must be actively respected.
Cost	Free and open-source. The cost is discipline: never run compression on the main thread.

DR-04 sharp for image compression

Decision	sharp (backed by libvips) as the image-processing library.
Alternatives considered	ImageMagick / GraphicsMagick, or a pure-JS library like Jimp.
Why	sharp is the fastest option because libvips runs native C off-thread and releases the event loop; it supports JPEG, PNG, WebP, AVIF, GIF, TIFF, and includes safeguards like pixel limits.
Trade-off accepted	A native dependency, so the deployment environment must support the compiled binary.
Cost	Free and open-source. Native builds are handled automatically on the target free hosts.

DR-05 BullMQ on Redis for the queue

Decision	BullMQ, a Redis-backed queue library, as the job basket.
Alternatives considered	Kafka (event streaming) or AWS SQS (managed cloud queue).
Why	BullMQ is right-sized for a portfolio project: easy to run, strong TypeScript support, and built-in retry-with-backoff and dead-letter handling. Kafka is over-scaled for this; SQS locks you into AWS.
Trade-off accepted	Redis-backed queues do not match Kafka's throughput ceiling — irrelevant at this scale, but worth acknowledging.
Cost	Free — runs on self-hosted or free-tier Redis, the same instance used for caching.

DR-06 Redis for both queue and live status ("two hats")

Decision	Use one Redis instance for the BullMQ queue and for caching pollable job status.
Alternatives considered	A separate cache store, or serving status directly from Postgres on every poll.
Why	Both uses suit Redis's in-memory speed, and status is read-heavy and ephemeral. One store for two matched jobs keeps the stack lean and reduces operational surface.
Trade-off accepted	Both responsibilities share one instance's availability — a single point to keep healthy.
Cost	Free, and cheaper than running two stores or hammering Postgres with poll traffic.

DR-07 PostgreSQL for durable job records

Decision	Postgres for the small, durable record of each job.
Alternatives considered	MongoDB / a document store, or Redis-only with no durable layer.
Why	The data is simple and structured, and Postgres is the reliable, transactional default that is easy to defend in an interview and leaves room to grow into accounts or history later.
Trade-off accepted	Arguably heavier than strictly needed — Redis alone could nearly carry this app.
Cost	Free tiers (Supabase, Neon, Railway). The mild over-provisioning buys defensibility and future room.

DR-08 No user accounts; anonymous, expiring job IDs

Decision	No authentication; issue an unguessable job ID at upload and auto-expire results.
Alternatives considered	Full user accounts with login and saved compression history.
Why	The product's value is instant, zero-friction utility. Auth would add friction, a security surface, and PII handling for no user benefit. Anonymity plus expiry is also a genuine privacy feature.
Trade-off accepted	No cross-device history and no per-user features — an acceptable loss for this tool.
Cost	Free, and cheaper in liability: no passwords, no PII, no auth vulnerabilities to defend.

DR-09 Pre-signed / direct-to-storage uploads (at scale)

Decision	Have clients upload raw bytes directly to object storage, not through the API server.
Alternatives considered	Route every upload through the API server, which then forwards bytes to storage.
Why	Keeps heavy image bytes off the API server so it stays responsive under load — separating the control plane from the data plane.
Trade-off accepted	More client-side complexity (requesting an upload URL first) and storage-permission setup.
Cost	Free on storage free tiers. Noted as a scaling step to reason about, not necessarily build in phase 1.

DR-10 SSIM-validated quality with format selection

Decision	Validate output with SSIM against the original and serve WebP/AVIF based on the Accept header.
Alternatives considered	Hardcode a single JPEG quality value and always output JPEG.
Why	Measuring similarity guarantees visual fidelity instead of assuming "smaller is fine," and modern formats cut 25–50% more at equal perceived quality. This is the project's distinctive depth.
Trade-off accepted	Extra compute per image (SSIM calculation, possibly multiple encode passes).
Cost	Free (open-source tooling); the cost is a modest amount of additional worker CPU per image.

DR-11 Ghostscript for PDF compression (extension)

Decision	Use Ghostscript to downsample embedded images and strip metadata when PDF support is added.
Alternatives considered	A commercial PDF SDK, or attempting Word↔PDF conversion instead.
Why	PDF compression mirrors the existing pipeline (file in, shrink, file out) and reuses the queue and worker model. Ghostscript is the mature free tool; commercial SDKs cost money and conversion is a separate, harder problem.
Trade-off accepted	Ghostscript is an external system binary the server shells out to — not a tidy npm package — so the host must have it installed.
Cost	Free and open-source. The cost is one deployment dependency to provision on the host.

4. Architecture at a Glance

The system separates the **control plane** (lightweight coordination — accepting requests, tracking jobs) from the **data plane** (the heavy work — moving and compressing image bytes). This separation is the single most important structural idea in the design.



Reading the diagram: the browser talks only to the API server and to object storage. The API server enqueues jobs into Redis and records them in PostgreSQL. Workers pull from the Redis queue, read and write image bytes to object storage, and update live status back in Redis. Nothing heavy ever blocks the API server.

5. Component Breakdown

The API server (Node.js + TypeScript)

Node is a strong fit for this layer because the API server's work is **I/O-bound**: accepting requests, issuing job IDs, reading and writing small records. Node's event loop excels at exactly this kind of work. The critical rule: the API server must **never** run compression on its main thread, because compression is CPU-bound and Node is single-threaded — doing so would block every other request. Compression lives in separate worker processes.

The compression function (sharp)

sharp is a high-performance, open-source image library backed by libvips (native C). It handles JPEG, PNG, WebP, AVIF, GIF and TIFF, and it releases Node's event loop because the heavy work runs in native code off-thread. The compression logic is written as a single self-contained function — conceptually `processImage(job)` — that takes a job and produces a compressed file. In the simple version the route calls it directly; in the production version the worker calls the very same function. This is the "design the seam first" principle in action.

The queue (BullMQ on Redis)

BullMQ is a mature, TypeScript-friendly queue library that uses Redis as its underlying store. It is the "basket" of tickets. It is chosen over heavier options because it is right-sized: Kafka is for high-throughput event streaming, and SQS ties you to AWS. BullMQ also provides retry-with-backoff and dead-letter handling out of the box, which the reliability section relies on.

The worker pool

Workers are separate processes whose only job is to pull tickets from the queue and run the compression function. Because they are separate from the API server, heavy CPU work never affects request responsiveness. Because they all pull from the same queue, adding more workers to handle more load requires **no code change** — this is what makes horizontal scaling cheap.

Object storage

Raw uploads and compressed outputs live in object storage (S3-compatible; MinIO can run locally for free). Files are referenced by ID rather than passed around in memory. Storing by ID from the start means that when scaling up, a local folder can be swapped for real object storage with almost no other change.

6. The Data Layer — Redis & PostgreSQL

This service stores surprisingly little. Per job, we keep an ID, a status, the original and compressed sizes (to show "saved 62%"), and a timestamp. There is no rich web of relationships. Two stores are used, each matched to the nature of the data.

Redis — the fast, ephemeral layer (wearing two hats)

Redis is an in-memory store, ideal for fast access and short-lived data. In this design it does two well-matched jobs:

- **Hat 1 — the queue.** BullMQ stores its pending job tickets in Redis. This is the "basket" workers pull from; it is fast and safe for multiple workers to read simultaneously.
- **Hat 2 — live status.** The status the frontend repeatedly polls ("processing... processing... done") is ephemeral and read-heavy — exactly what an in-memory cache is for. It auto-expires after an hour since no history is kept.

Why this is a strength, not a shortcut

One piece of infrastructure serves two purposes that both suit its strengths, which keeps the stack lean.

Interview line: "Redis serves double duty — it backs my job queue and caches live status — so I run one fast store instead of two."

PostgreSQL — the durable record

Postgres holds the durable job record — the data that should survive and could be queried later. On pure technical fit this app is light enough that Redis alone could nearly carry it, so Postgres is chosen partly for a **defensible** reason: it is the reliable, transactional default, and if the product ever grew accounts or history, the relational model is already there to extend.

The database interview answer

"Live job status lives in Redis because it is ephemeral and read-heavy. Durable job records live in PostgreSQL because it is a reliable, transactional default and leaves room to grow. I split the data by whether it is ephemeral or durable." This demonstrates reasoning about data lifetime and current-needs-versus-future-flexibility — the senior instinct interviewers probe for.

No accounts — a deliberate product decision

The tool's value is instant, zero-friction utility: land, drop an image, get a smaller one, leave. An account gate would add friction, a security surface, and PII-handling burden for no user benefit. Removing accounts also removes real liability — no passwords, no PII, no auth vulnerabilities. Users still retrieve their result via an **unguessable anonymous job ID** issued at upload; the result auto-expires and nothing is retained.

7. Preserving Image Quality

Compression is only impressive if the output still looks good. This is the domain-specific depth that distinguishes the project from "I wired some services together."

Quality is a knob, not a constant

JPEG quality around 80 with mozjpeg enabled is a strong, near-visually-lossless default with large size savings. But a good system does not hardcode a single value blindly.

Format often matters more than the quality slider

Serving WebP or AVIF instead of JPEG frequently yields 25–50% smaller files at the same perceived quality. A sophisticated approach inspects the browser's Accept header and serves the best format the client supports, falling back to JPEG.

Measure quality — don't guess (SSIM)

SSIM (Structural Similarity) compares the compressed image to the original and produces a similarity score. Computing it lets the system **verify** that output stays above a quality threshold rather than assuming "smaller is fine." A refined version steps quality down until SSIM approaches the threshold — capturing maximum size savings while guaranteeing visual fidelity. Mentioning SSIM signals that you think about validation, not just output.

Free wins

- Resize to sane maximum dimensions; never upscale.
- Strip EXIF/metadata to shed bytes.
- Preserve the color profile — dropping it shifts colors, a subtle but important detail.

8. Handling Scale — The 10,000-Users Question

"How would you handle 10,000 users overnight?" is a classic intern-interview question. It does not test whether you built for 10,000 users; it tests whether you can **reason about bottlenecks**. The method: walk through the system, find what breaks first, name the relief, then find the next bottleneck. Bottleneck by bottleneck.

#	What breaks first	How you relieve it
1	Compression blocks the server — CPU work strangles the event loop.	Queue + worker model: uploads are accepted instantly; processing happens in the background.
2	One worker can't drain the backlog.	Run more workers in parallel (horizontal scaling). Trivial because they share one queue — zero code change.
3	The API server saturates handling upload bytes.	Pre-signed URLs: raw images go straight to storage, never touching the app server, which only coordinates.
4	The database is hammered by status polling.	Serve status from the Redis cache, or push updates via websockets/SSE instead of polling.

The key insight to state out loud: this is not a recital of features — it is reasoning through failure points **in order**. Even if only the first two relief steps are actually built, being able to reason through all four is what scores. The async architecture **is** the scaling answer: the queue absorbs traffic spikes (backpressure), and workers drain it at their own pace.

9. Security, Validation & Abuse Prevention

An open, accountless endpoint is convenient for users and attractive to abuse. Spotting the downside of your own design decision — and addressing it unprompted — is exactly what interviewers notice.

- **Never trust the file.** Validate that an upload is actually an image by checking its magic bytes, not its file extension.
- **Cap file size.** Reject oversized uploads before they consume memory or worker time.
- **Guard against decompression bombs.** A tiny file can expand to gigabytes in memory; sharp's `limitInputPixels` defends against this.
- **Rate-limit by IP.** Since there are no accounts, IP-based rate limiting prevents a single client from hammering the endpoint.
- **Minimize retained data.** Auto-expiring files and no stored PII is the right posture — reducing sensitive data held is instinctively correct, especially at a bank.

The interview line

"Because the service is open and accountless, I rate-limit by IP and validate uploads — real-image checks, size caps, and pixel limits to stop decompression bombs. I deliberately retain nothing, which shrinks both the security and privacy surface."

10. Failure Handling & Reliability

Production systems are judged on the non-happy path. These are the reliability concerns to build for and be ready to discuss.

- **Idempotency.** If the same upload event fires twice, the system should not compress twice. Keying jobs by a content hash makes repeated events harmless.
- **Worker death mid-job.** If a worker crashes while processing, the job must be retried. BullMQ provides retry-with-backoff and a dead-letter queue for jobs that repeatedly fail.
- **Backpressure.** When 10,000 uploads arrive at once, the queue absorbs the spike and workers drain it at their own pace. This is the core reason the async design wins.
- **Observability.** Track compression ratio, processing latency, queue depth, and failure rate. Knowing *what* you would monitor signals production maturity even before dashboards exist.

11. Staged Build Plan

Build in layers so there is always something working and demoable. Crucially, build the simple version first so you feel the blocking problem yourself — then the queue-worker design lands as an obvious solution to a real problem you experienced. That arc is the most convincing thing you can describe in an interview.

Phase	What you build	What it teaches / the talking point
1	Simple synchronous compressor: upload → sharp → download link. Compression already isolated in its own function; files stored by ID.	Proves the core works. Establishes clean seams so the next phase is a swap, not a rewrite.
2	Move compression into a worker + BullMQ/Redis queue. API returns a job ID; frontend polls.	"I built it naively, watched it block under concurrent uploads, and re-architected around a queue." The strongest story.
3	Deploy to a free host with a live link. Add Postgres for durable records.	A real, shareable, demoable project. Reason about ephemeral vs durable data.
4	Add quality validation (SSIM) and format selection (WebP/AVIF via Accept header). Add rate limiting + input validation.	Domain depth + security awareness — the distinctive parts.
5	Stop building; be ready to <i>reason through</i> pre-signed URLs, more workers, and caching as next steps.	Maximum interview mileage without infinite building.

Discipline note

Do not let "make it scale" become an excuse never to finish. A deployed, simple, working compressor with clean seams and a clear scaling story beats a half-built distributed system every time. Ship the simple thing first, then widen.

12. Free Tooling & Hosting

The entire system runs on free options. The only trade-off is that free tiers may sleep when idle or impose limits — perfectly fine for a portfolio project demoed in interviews.

Concern	Free option
Image compression	sharp (open-source, backed by libvips)
Queue	BullMQ on a self-hosted or free-tier Redis
Live status / cache	Redis (same instance)
Durable records	PostgreSQL free tiers — Supabase, Neon, or Railway
Object storage	MinIO (S3-compatible, runs locally) or a free tier
Hosting	Render, Railway, or Fly.io free tiers

13. Extension — Adding PDF Compression

Once the image compressor is fully built and deployed, PDF compression is the right next feature — because it is the **same shape** as what already exists: a file comes in, gets shrunk, and goes out. It reuses the same queue, worker model, and anonymous-job-ID pattern. This generalizes the project from "image compressor" to "file compressor" — a coherent expansion, not a grab-bag.

Why PDF compression fits (and Word↔PDF does not)

PDF *compression* matches the existing pipeline. By contrast, Word↔PDF *conversion* is a separate, harder problem — faithful layout reproduction, heavy tooling, and disappointing reverse conversion — and the good tools cost money. PDF compression stays on-theme and free.

How PDF compression works

A PDF is often large because of the **images embedded inside it**. Compressing a PDF therefore largely means downsampling and re-compressing those embedded images, plus stripping unused fonts and metadata. `sharp` does not handle PDFs; the standard free tool is **Ghostscript**, a mature open-source program the server invokes to rewrite the PDF at a lower quality preset.

The one operational wrinkle

Ghostscript is an external system binary the server shells out to, not a tidy npm package like `sharp`. That means the deployment host must have Ghostscript installed — a small but real operational consideration worth noting.

The quality parallel

Ghostscript exposes preset levels (roughly: screen, ebook, printer — increasingly less aggressive). Choosing among them is the same quality-versus-size trade-off already reasoned about for JPEG quality and SSIM, so PDF support **reinforces** the project's strongest talking point rather than introducing a disconnected one.

The design detail: a pluggable pipeline

The worker inspects the file type and routes to the right handler: images go to `sharp`, PDFs go to Ghostscript. Adding a new file type becomes "add a handler," not "rewrite the system." That extensibility story is itself worth describing in an interview.

Sequencing discipline

Do not build both at once. Ship images fully first, then generalize the pipeline to add PDF as a second handler. Depth first, then widen — the same discipline applied throughout.

14. Interview Talking Points

A condensed set of lines that show reasoning rather than buzzwords. Each maps to a decision made in this document.

- **The re-architecture arc:** "I built it naively, measured it blocking under concurrent uploads, and re-architected around a queue and worker pool."
 - **Control plane vs data plane:** "The API server coordinates; it never handles heavy compression itself, so it stays responsive."
 - **Node's constraint:** "Compression is CPU-bound and Node is single-threaded, so it runs in separate workers, never on the request thread."
 - **Data lifetime:** "Ephemeral status in Redis, durable records in Postgres — I split storage by whether the data needs to survive."
 - **Product judgment:** "No accounts — the value is zero-friction utility, and skipping auth removes friction, PII, and a security surface."
 - **Quality validation:** "I verify output with SSIM rather than assuming smaller is acceptable, and pick WebP/AVIF by the browser's Accept header."
 - **Scaling by reasoning:** "Add workers for throughput, pre-signed URLs to offload the server, caching for status reads — bottleneck by bottleneck."
 - **Spotting your own downside:** "Because it is open and countless, I rate-limit by IP and validate uploads against bombs and non-images."
 - **Extensibility:** "The worker routes by file type — images to sharp, PDFs to Ghostscript — so adding a format is adding a handler."
-

Above all: depth of reasoning about your own trade-offs beats surface-level buzzword coverage. Ship the simple thing, understand every choice, and be honest about what you built versus what you would build next.